

# FraudFlow

Detecção de fraude em transações de cartão com Arquitetura Medallion e BigQuery ML.

1.296.675

TRANSAÇÕES PROCESSADAS

7.506

FRAUDES NO PERÍODO

0,58%

DA BASE — O DESAFIO

# Do CSV bruto ao modelo, tudo dentro do BigQuery



Cada camada só depende da anterior. **Se a Silver estiver errada, refaz a partir da Bronze — sem baixar nada de novo.** É o que torna o pipeline auditável e refazível.

# A camada que não conserta nada — de propósito

- **Carga direta do Cloud Storage**

O BigQuery lê o arquivo sozinho, em paralelo. Nenhum dado passa pelo notebook — e carga em lote é gratuita.

- **As 23 colunas guardadas como texto**

Converter na entrada faria a carga falhar por causa de uma linha ruim. Como texto, nada se perde, e a decisão de corrigir passa a ser da Silver.

- **Duas colunas de linhagem: source\_file e ingestion\_timestamp**

Permitem provar de qual arquivo cada linha veio — a evidência de que o conjunto de teste ficou de fora.

- **Sem particionamento**

A carga aconteceu de uma vez só: particionar por data de ingestão criaria uma única partição gigante, sem ganho nenhum.

|                                                   |                                                  |                                                        |
|---------------------------------------------------|--------------------------------------------------|--------------------------------------------------------|
| <div>1.296.675</div> <div>LINHAS CARREGADAS</div> | <div>25</div> <div>COLUNAS (23 + LINHAGEM)</div> | <div>0 bytes</div> <div>TRAFEGARAM PELO NOTEBOOK</div> |
|---------------------------------------------------|--------------------------------------------------|--------------------------------------------------------|

# Limpeza que se prova com número

- **Tipagem com `SAFE_CAST` e `SAFE.PARSE`**

Texto vira número, data e timestamp. O que não converte vira nulo em vez de derrubar a consulta — o descarte fica controlado e contável.

- **Resolvida uma inconsistência de sete anos no dataset**

As duas colunas de tempo discordavam em exatamente sete anos-calendário. Calcular idade pela errada deixaria todo cliente sete anos mais novo, sem erro aparecer.

- **Prefixo `fraud_` removido dos 693 comerciantes**

Artefato do gerador, presente também em transações legítimas. Deixá-lo convida a um filtro acidental.

- **Nove regras de sanidade e deduplicação global**

Coordenadas na faixa válida, nascimento anterior à compra, rótulo apenas 0 ou 1. A deduplicação roda em SQL, com a tabela inteira à vista.

- **Dados pessoais deixados de fora**

Nome, rua, CEP, gênero e profissão não entram. Não trazê-los torna a escolha verificável.

|            |                  |                                 |
|------------|------------------|---------------------------------|
| 0          | 0                | 21/06/2020                      |
| DUPLICATAS | PREFIXO RESTANTE | FIM DA JANELA — HOLDOUT INTACTO |

# Duas saídas, porque são dois consumidores

## GOLD.ML\_INPUT — PARA O MODELO

- **O contrato, em forma de tabela**

Define exatamente quais campos o modelo consome. Em outubro, o evento do streaming terá esses mesmos campos.

- **Uma linha por transação**

10 campos, mais o rótulo e o identificador — usado para juntar a resposta verdadeira depois da predição.

**1.296.675**

LINHAS

## GOLD.FRAUD\_ANALYTICS — PARA O NEGÓCIO

- **Agregação por categoria, hora e estado**

Taxas de fraude, ticket médio e volume. É o que um painel consome e o que vira slide.

- **Uma linha por grupo**

Agrupar também por dia deixaria quase uma linha por transação, e a taxa de fraude viraria 0% ou 100%.

**12.122**

LINHAS AGREGADAS

**Nenhuma feature do modelo é calculada aqui.** Elas ficam no TRANSFORM

# As features nascem dentro do modelo

## AS NOVE FEATURES

- **hour, day\_of\_week, is\_night**

Extraídas do timestamp.

- **age**

Idade na data da compra, com ajuste para o aniversário que ainda não passou no ano.

- **distance\_km**

Haversine entre cliente e comerciante — distância sobre a superfície curva da Terra.

- **amount, log\_amount, category, city\_pop**

Valor, sua escala comprimida, categoria e porte da cidade.

## POR QUE DENTRO DO MODELO

O TRANSFORM é salvo **junto com o modelo**.

Se as features fossem calculadas na Silver, existiriam **duas implementações** — uma em SQL e outra na API — livres para divergir. Bastaria alguém escrever **hora <= 6** de um lado e **hora < 6** do outro para o modelo passar a receber, em produção, features diferentes das que viu no treino.

# Recall e precision, não acurácia

| MODELO            | RECALL | PRECISION | F1    | AUC   |
|-------------------|--------|-----------|-------|-------|
| v1 · logística    | 0,867  | 0,041     | 0,079 | 0,949 |
| v2 · boosted tree | 0,969  | 0,298     | 0,456 | 0,999 |

No período de validação — os 20% finais por tempo

Responder “não é fraude” sempre daria 99,41% de acurácia — mais que os 98,63% do nosso modelo. É por isso que acurácia não serve aqui.

MATRIZ DE CONFUSÃO · V2

De cada 100 fraudes, o modelo captura **97**.

Em troca, marca **1,36%** das transações legítimas para revisão — não para bloqueio.

**Honestidade:** o dado é sintético e as separações são limpas demais. A madrugada tem 18× mais fraude — e o modelo elegeu is\_night como feature mais decisiva, com folga de 3×. Ele redescobriu a regra do simulador.